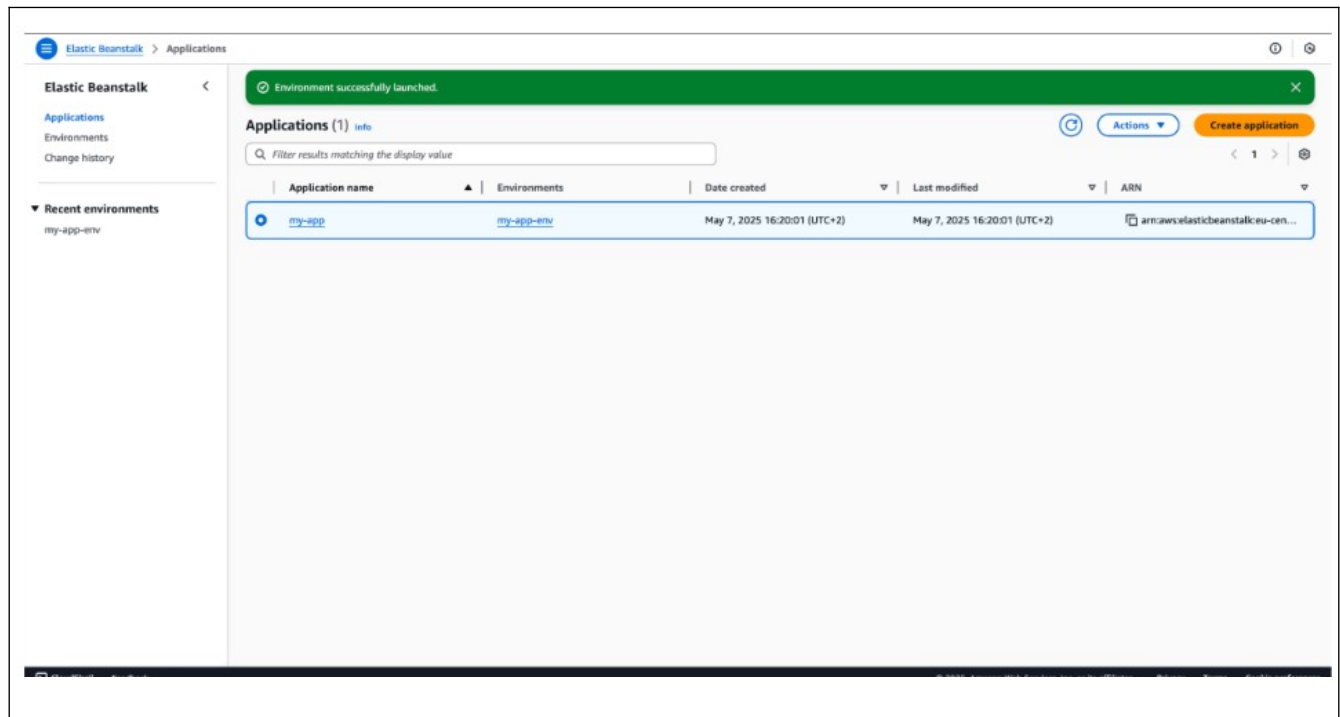


Step 1: Initialize the Application in AWS Elastic Beanstalk

Action: Navigate to the AWS Elastic Beanstalk console and access the **Applications** dashboard. As demonstrated in the figure, a new logical application named my-app has been created. Consequently, Elastic Beanstalk automatically provisioned an initial environment named my-app-env to host the application version.

Educational Insight & Security Considerations: It is important to understand that in AWS Elastic Beanstalk, an **Application** acts as a high-level conceptual container (similar to a folder) that holds versions of your deployable code and configurations. The **Environment** is the actual provisioned infrastructure (EC2 instances, Load Balancers, Security Groups) running a specific application version. From a **Cyber Security** perspective, deploying via Elastic Beanstalk allows us to leverage AWS Managed Services, reducing the operational burden of patching the underlying OS. However, it is critical to implement the principle of least privilege immediately. For future steps, we must ensure that the IAM instance profile assigned to the EC2 instances within this environment has only the minimum necessary permissions to perform its intended function, and that the associated Security Groups allow traffic only on required ports.



Step 2: Environment Capacity and Instance Configuration

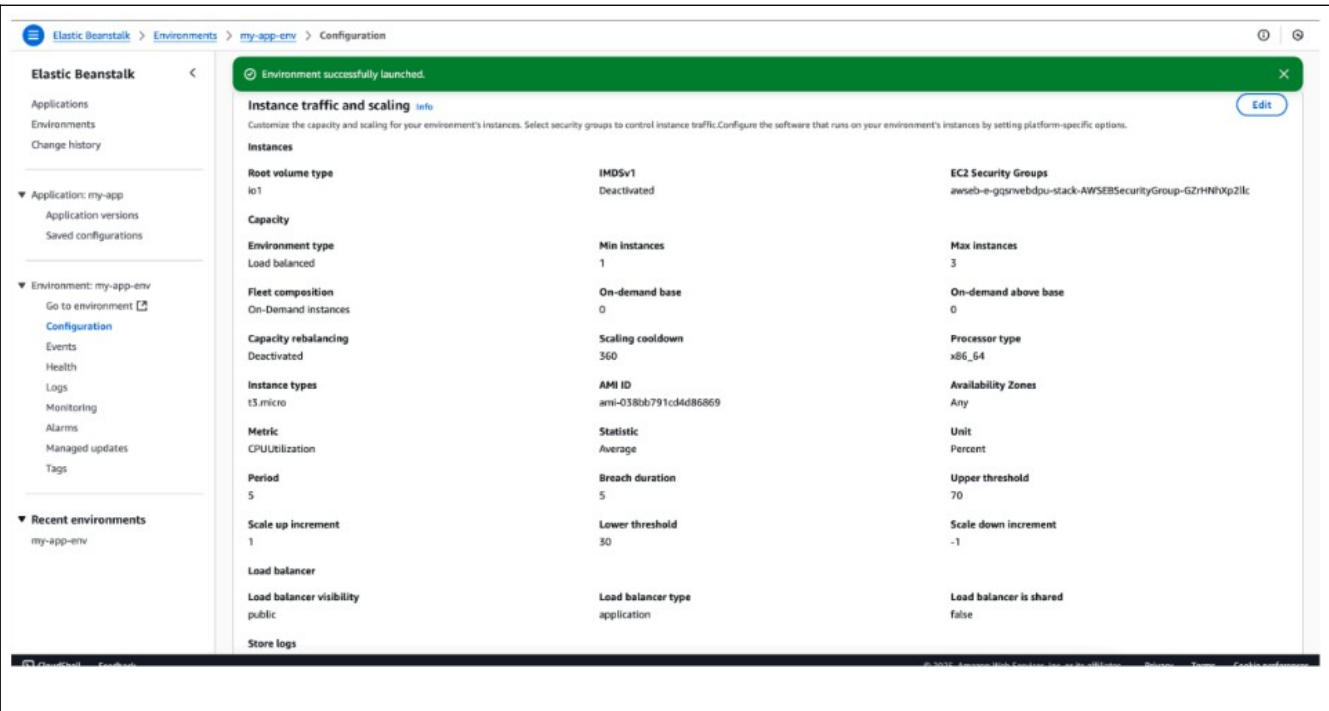
Action: We now navigate to the **Configuration** section of the my-app-env environment and access the **Capacity** settings. As illustrated in the console screenshot, the environment has been configured to use the **t3.micro** EC2 instance type. This instance family is part of the AWS Free Tier, making it cost-effective for development and testing.

Furthermore, we confirmed that the **Environment type** is set to **Load balanced**. This is a critical architectural choice even for a simple application, as it ensures that traffic is distributed across multiple instances, enhancing reliability.

Educational Insight & Security Considerations (DevSecOps Perspective):

From a **Cloud Architecture and Security** standpoint, configuring capacity requires balancing performance, cost, and risk:

- 1. **Cost and Risk Mitigation:** By selecting small instances (t3.micro) and initially setting the minimum instance count to one, we practice responsible cloud spending. Crucially, as seen in the screenshot, we have defined a **Maximum instance** limit of 3. This is a vital security best practice to protect against unintended cloud spend or unexpected traffic surges (such as a simple DDoS attack or a rapid scaling event) from consuming too many resources and generating massive unexpected costs.
- 2. **Instance Metadata Security:** The screenshot shows **IMDSv1** deactivated. While not shown directly as "active" or "inactive" in the main view without clicking edit, it is standard practice to enforce the use of Instance Metadata Service Version 2 (IMDSv2) for all new EC2 instances. IMDSv2 provides protection against specific server-side request forgery (SSRF) vulnerabilities that could otherwise be used to compromise IAM credentials.



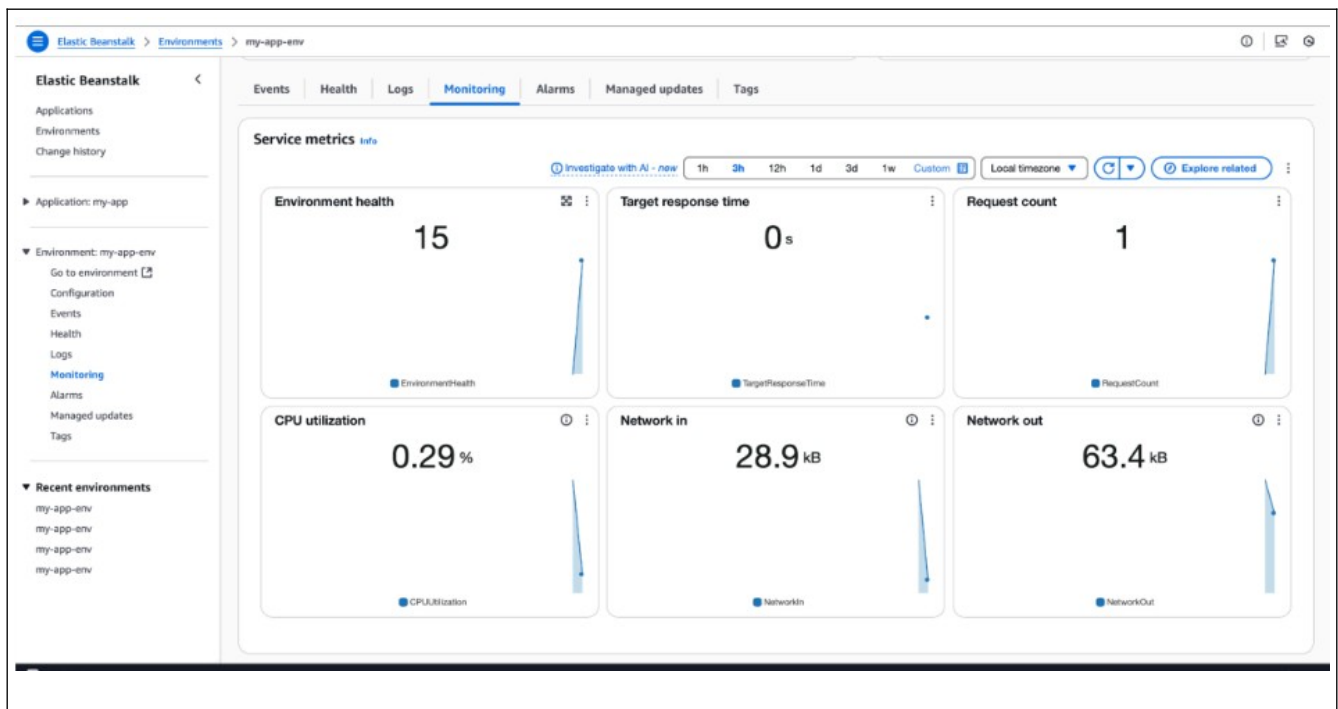
Step 3: Application Monitoring and Performance Observability Action:

To ensure the operational health of the application, I navigated to the **Monitoring** tab within the Elastic Beanstalk dashboard. This view provides real-time visibility into the environment's performance metrics. As shown in the screenshot, I am tracking key indicators such as **CPU Utilization** (\$0.29\%\$), **Request Count**, and **Target Response Time**. These metrics are integrated with Amazon CloudWatch to provide a continuous stream of performance data.

Educational Insight & Security Considerations (The "Cybersecurity Angle"):

From a **Security Operations (SecOps)** perspective, monitoring is not just about performance; it is a primary line of defense:

1. **Anomaly Detection:** Monitoring **CPU Utilization** and **Network In/Out** is essential for detecting potential security breaches. For instance, a sudden, unexplained spike in CPU could indicate unauthorized crypto-mining or a malware process, while unusual network egress might signal data exfiltration.
2. **DDoS Mitigation:** Tracking **Request Count** and **Target Response Time** allows for the early detection of Application Layer (Layer 7) DoS attacks. If the request count spikes while response time increases, it triggers an immediate investigation into the traffic source.
3. **Audit Readiness:** In a production environment, these metrics (along with the **Logs** tab seen in the menu) are vital for forensic analysis after a security incident to determine the timeline and impact of the event.



Step 4: Configuring Auto Scaling Policies and Triggers

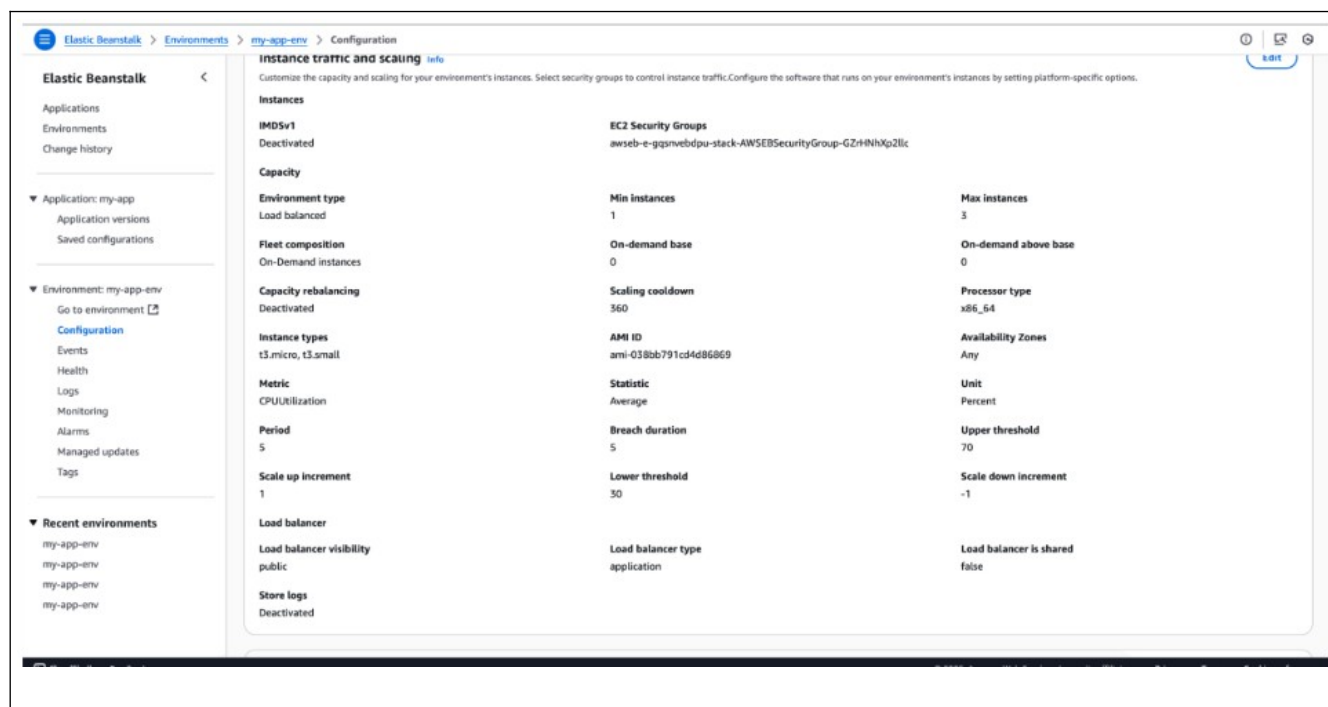
Action: In this step, I configured the **Auto Scaling** group settings within the environment's **Configuration** dashboard. As captured in the screenshot, the environment is set with a **Minimum instance count of 1** and a **Maximum of 3**. To automate the scaling process, I established a **Scaling Trigger** based on **CPU Utilization**. Specifically, if the average CPU load exceeds the **Upper threshold of 70%** for a sustained period, the system will automatically provision an additional instance to handle the increased traffic.

Educational Insight & Security Considerations (DevSecOps Perspective):

From an **Availability and Security** standpoint, Auto Scaling is a core pillar of the **AWS Well-Architected Framework**:

1. **Availability vs. Cost:** By setting a "Max instances" limit, we ensure the application can scale out to maintain availability during a surge (preventing a denial-of-service condition due to resource exhaustion), while simultaneously implementing a "financial circuit breaker" to prevent runaway costs.

2. **Self-Healing Architecture:** Auto Scaling doesn't just respond to traffic; it also acts as a health mechanism. If an EC2 instance becomes unhealthy or fails a health check, the Auto Scaling group will automatically terminate it and launch a new one, maintaining the "Min instances" requirement.
3. **Security Response:** In a cybersecurity context, scaling triggers can sometimes be the first indicator of a resource-heavy exploit attempt. Monitoring these scaling events is vital for identifying whether growth is organic or due to malicious activity.



Step 5: Application Deployment Verification and Public Access

Action: In the final phase of this lab, I performed an end-to-end verification of the deployment. By navigating to the environment's unique URL generated by AWS Elastic Beanstalk (as seen in the browser's address bar), I successfully accessed the live web application. The "Congratulations" landing page confirms that the Python platform is correctly initialized and the environment is healthy and reachable over the public internet.

Educational Insight & Security Considerations (Final Review):

While the application is successfully running, a **Cloud Security** specialist would evaluate the following "Day 2" operations:

1. **SSL/TLS Encryption (HTTPS):** In the screenshot, the browser indicates a "Not Secure" connection because the default Elastic Beanstalk URL uses HTTP. For a production environment, the next critical security step would be to associate a custom domain name and provision an **SSL/TLS certificate via AWS Certificate Manager (ACM)** to ensure data in transit is encrypted.
2. **Network Access Control:** The successful access confirms that the Security Groups associated with the Load Balancer are correctly configured to allow inbound traffic on port 80. To further

harden the application, we could implement a **Web Application Firewall (AWS WAF)** to protect against common web exploits such as SQL injection or Cross-Site Scripting (XSS).

3. **DNS Management:** Using Amazon Route 53 would allow us to create a user-friendly alias for this long Elastic Beanstalk URL, further professionalizing the deployment.

